

NSF Workshop Report

Deming Chen, *Fellow, IEEE*, Vijay Ganesh, Weikai Li, Yingyan Lin, Yong Liu, Subhasish Mitra, *Fellow, IEEE*, David Z. Pan, *Fellow, IEEE*, Ruchir Puri, *Fellow, IEEE*, Jason Cong, *Fellow, IEEE*, and Yizhou Sun, *Senior Member, IEEE*

Report for NSF Workshop on AI for Electronic Design Automation

Abstract

This report distills the discussions and recommendations from the NSF Workshop on AI for Electronic Design Automation (EDA), held on December 10, 2024 in Vancouver along-side NeurIPS 2024. Bringing together experts across machine learning and EDA, the workshop examined how AI—spanning large language models (LLMs), graph neural networks (GNNs), reinforcement learning (RL), neurosymbolic methods, etc.—can facilitate EDA and shorten design turnaround. The workshop includes four themes: (1) AI for physical synthesis and design for manufacturing (DFM), discussing challenges in physical manufacturing process and potential AI applications; (2) AI for high-level and logic-level synthesis (HLS/LLS), covering pragma insertion, program transformation, RTL code generation, etc.; (3) AI toolbox for optimization and design, discussing frontier AI developments that could potentially be applied to EDA tasks; and (4) AI for test and verification, including LLM-assisted verification tools, ML-augmented SAT solving, security/reliability challenges, etc. The report recommends NSF to foster AI/EDA collaboration, invest in foundational AI for EDA, develop robust data infrastructures, promote scalable compute infrastructure, and invest in workforce development to democratize hardware design and enable next-generation hardware systems. The workshop information can be found on the website <https://ai4eda-workshop.github.io/>

Index Terms—AI for electronic design automation, chip design, workshop report, physical design, high-level synthesis, logic-level synthesis, test and verification, large language model, graph neural network, reinforcement learning (RL), neurosymbolic AI.

I. Executive Summary

This report summarizes the key discussions and recommendations from the NSF Workshop on AI for Electronic Design Automation (EDA), held

alongside the NeurIPS 2024 conference in Vancouver, Canada. The workshop brought together leading researchers and practitioners from both the machine learning (ML) and EDA communities to address the urgent need for AI-driven automation in chip design.

The central challenge in hardware design is the increasing complexity of modern chips. Domain-specific accelerators (DSAs) offer substantial performance and energy efficiency gains, but their design demands deep hardware expertise, leading to long turnaround times and hindering rapid innovation. AI techniques (such as Large Language Models), in combination with traditional EDA tools, have the potential to offer a transformative solution to democratize hardware design, making it domain-specific, cheaper, and far more effective.

The workshop explored four key themes:

■ **AI for Physical Synthesis and Design for Manufacturing (DFM):** This theme addressed the challenges in translating a chip design into a manufacturable product, including multi-objective optimization, complex constraints, and long turnaround times. Current AI efforts leverage Reinforcement Learning (RL) for placement, surrogate models for analysis, LLMs for rule creation, and generative AI for design acceleration. Key show-stoppers include generalization issues across designs and nodes, scalability limitations, data quality and availability, and the significant computational resources required for AI models. Opportunities lie in fostering cross-community collaboration, developing open-source frameworks and datasets, and exploring hybrid AI approaches that combine deep learning with traditional methods. Recommendations to NSF include supporting foundational AI research for EDA, encouraging open-source initiatives, promoting scalable compute infrastructure, and investing in workforce development.

■ **AI for High-Level and Logic-Level Synthesis (HLS and LLS):** This section focused on automating the translation of high-level behavioral descriptions into gate-level specifications. The core problem

Deming Chen is with University of Illinois Urbana-Champaign, Urbana, IL 61801 USA.

Vijay Ganesh and Yingyan Lin are with Georgia Institute of Technology, Atlanta, GA 30332 USA.

Weikai Li, Jason Cong (corresponding author), and Yizhou Sun (corresponding author) are with the University of California at Los Angeles, Los Angeles, CA 90095 USA (e-mail: cong@cs.ucla.edu; yzsun@cs.ucla.edu).

Yong Liu is with Cadence Design Systems, Inc., San Jose, CA 95134 USA.

Subhasish Mitra is with Stanford University, Stanford, CA 94305 USA.

David Z. Pan is with the University of Texas at Austin, Austin, TX 78712 USA.

Ruchir Puri is with IBM, Poughkeepsie, NY 12601 USA.

Digital Object Identifier 10.1109/MCAS.2026.3662307

Date of current version: 1 April 2026

is the manual, error-prone, and time-consuming nature of traditional synthesis, exacerbated by the labor disparity in hardware design. AI can automate design decisions, predict performance outcomes, and enhance design space exploration. Existing efforts include HLS coupled with deep learning optimization (HLS+DL), using Graph Neural Networks (GNNs) and LLMs for performance prediction and pragma insertion, and more ambitious natural language to RTL (NL2RTL) generation. Showstoppers include the difficulty in adapting AI models to toolchain changes and new designs (domain shift), and the critical need for high-quality, labeled training data. Significant opportunities exist for the AI community to tackle novel problems in multi-modality fusion, domain/task transfer, and optimization, while the EDA community can leverage AI for accelerated pipelines, synthetic data generation, and black-box optimization.

- **AI Toolbox for Optimization and Design:** This section focused on challenges raised to AI from chip design. Existing AI techniques rely heavily on well-maintained offline data, a clearly defined objective function, and an offline evaluation. Key recommendations for NSF include the following. Generalization capability and uncertainty estimation become even more critical in chip design. Reinforcement learning and black-box optimization can lead to more effective and efficient search, which is beyond the traditional prediction tasks in general ML. LLM agents could serve as a very powerful tool by automatically putting long pipelines together and getting end-to-end solutions from data collection and tool calling to verification.
- **AI for Test and Verification:** This session explored the transformative role of artificial intelligence in test and verification within the domain of EDA. Key discussions included methods for learning robust design representations that generalize across multiple hardware design tasks, enhancing the adaptability and efficiency of AI-driven solutions. A focal point was an LLM-aided hardware design framework, which is uniquely enabled by G-QED (Generalized Quick Error Detection), a new verification technique for rapidly identifying logic bugs during chip verification. The workshop also addressed the evolving landscape of software development in the era of LLMs, emphasizing their application in fuzz testing and the novel use of LLMs for generating effective fuzzing programs. Another topic covered was security vulnerabili-

ties of AI-based techniques, as they are universally being employed in various testing tasks, and their unreliability can lead to nefarious system breaches. Finally, presenters showcased AI-based optimization strategies to efficiently solve complex SAT (satisfiability) verification problems, highlighting a path toward more scalable and accurate hardware verification processes. Collectively, the session emphasized the synergy between AI and EDA to enable faster, more intelligent design and design verification flows across the design lifecycle.

Overall, the workshop highlighted the immense potential of AI to revolutionize chip design. Realizing this potential requires a concerted effort to overcome challenges related to data availability, generalization, scalability, and integration. One common theme that came up was how to find optimized combinations of AI and non-AI approaches for large benefits—a truly cross-disciplinary approach to exploit the positives of each and avoid their negatives. Key recommendations for the NSF and the broader research community include:

- **Fostering cross-disciplinary collaboration** through shared datasets, benchmarks, open-source tools, and joint research programs.
- **Investing in foundational AI research** tailored for EDA, with a focus on explainability, scalability, and robustness.
- **Developing robust data infrastructures** to efficiently extract and manage EDA data, alongside establishing diverse and large-scale benchmark datasets.
- **Promoting scalable compute infrastructure** and encouraging the development of hybrid AI approaches that combine the strengths of AI with traditional EDA methodologies.
- **Investing in workforce development** to train a new generation of professionals at the intersection of AI and EDA.

By addressing these critical areas, the workshop aims to accelerate the adoption of AI in chip design, ultimately democratizing hardware innovation and enabling the creation of next-generation energy-efficient computing systems.

II. Introduction and Background

In the past few years, domain-specific accelerators (DSAs), such as Google's Tensor Processing Unit (TPU), have been shown to offer significant performance and energy efficiency over general-purpose CPUs. However, DSAs require deep hardware knowledge to achieve high performance and robustness. Unfortunately, there are far fewer hardware designers than software developers.

Leveraging AI techniques to further automate chip design becomes the key to meeting the need for rapid change in software development, which is also critical for democratizing hardware design and creating next-generation energy-efficient hardware. This workshop aims to bring researchers and practitioners from both machine learning and EDA communities to discuss how AI can address challenges in different stages of hardware design, promoting open benchmark datasets and open discussions to revolutionize chip design.

The workshop had 20 invited speakers and panelists covering four themes, and attracted 130+ attendees, on a wide variety of methods and problems in this area. The workshop covers four themes, including three key areas of EDA and the AI toolbox:

- AI for physical synthesis and design for manufacturing
- AI for high-level and logic-level synthesis
- AI toolbox for optimization and design
- AI for test and verification

The workshop includes a wide range of topics, including but not limited to:

- Graph Neural Networks for Performance, Power, and Area (PPA) prediction and optimization for all stages of the EDA pipeline, including HLS, RTL/ logic synthesis, and physical designs;
- LLM-based code analysis for PPA prediction and optimization at all stages of the EDA pipeline;
- Multi-modality ML models (e.g., LLM + GNN) for PPA prediction and optimization at all stages of the EDA pipeline;
- Domain and task transfer for HLS performance prediction with new kernels and new versions of EDA tools at different stages;
- ML methods to optimize circuit aging and reliability;
- ML for Design Technology Co-Optimization (DTCO);
- ML for analog, mixed-signal, and RF IC designs;
- Reinforcement learning for Design Space Exploration (DSE);
- LLM-based design generation;
- Interplay between AI-enabled design and verification;
- Active learning and importance sampling of design points;
- AI for compiler and code transformation;
- Benchmark datasets.

Based on the three key areas of EDA (AI for physical synthesis and design for manufacturing, AI for high-level and logic-level synthesis, and AI for test and verification), we formed round-table discussions to identify key challenges, SOTA approaches, and future directions.

Note that Theme 3 (AI toolbox for optimization and design) speakers and panelists participated in the other three themes' discussions.

III. AI for Physical Synthesis and Design for Manufacturing

A. Problem Description and Challenges

Physical synthesis and Design for Manufacturing (DFM) are essential stages in Electronic Design Automation (EDA), ensuring that a chip design can be efficiently and reliably produced. Modern chip design, with billions of transistors and advanced process nodes like 3 nm and 2 nm, presents significant challenges to these processes. Traditional manual and iterative approaches struggle to cope with the immense complexity and time constraints of modern chip development. Key challenges in this area are:

- Multiple objective optimizations: Physical design, DFM, and analog designs are multiple objective optimization problems.
- Complex constraints: Physical synthesis involves numerous constraints such as timing, power, density, and manufacturability, which grow exponentially with scaling to advanced nodes like 5nm, 3nm, and 2nm.
- DRC complexity: Design Rule Checking (DRC) involves thousands of rules, which must be checked and optimized. Many rules are negotiable with customers, further complicating the process.
- Combinatorial optimization: Placement, routing, and gate sizing are inherently combinatorial problems, often requiring significant computational resources to achieve near-optimal solutions.
- Analog and RFIC designs: While digital designs have well-established EDA abstractions and tools, analog and RF IC designs are still heavily manual and not scalable.
- Long turnaround times: Traditional methods result in weeks of delay for large designs due to iterative processes in routing, simulation, and verification.
- Manufacturing variations: Ensuring yield and robustness against process variations is increasingly difficult at advanced nodes, where small discrepancies can lead to significant defects.

AI/EDA has the potential to overcome these challenges.

B. Summary of Existing Efforts and Showstoppers

Existing Efforts:

- Reinforcement Learning (RL): Used for macro placement tasks [1], RL has shown some success

in improving placement quality. RL has also been shown for analog transistor sizing to meet tough design specifications [2].

- Surrogate models for design analysis: Faster and approximate models replace traditional analytical analysis for key design metrics such as timing, power, routability, etc., accelerating optimizations [3].
- LLM for rule creation: Large language models (LLMs) are being applied to automate DRC code writing and verification [4].
- Generative AI for physical design: Leverage generative AI models trained on optimized data to achieve a significant boost over traditional optimization methods, e.g., [5].
- Gradient-based method for physical design: model physical optimization problems as differentiable problems and optimize with gradient-based methods using deep learning frameworks on GPU [6], [7].
- Machine Learning (ML) for placement and routing, e.g., predictive tools assist in congestion analysis and timing closure [8].
- AI in DFM analysis: Techniques for hotspot detection [9], printed image [10] and process window predictions, and yield optimization to enhance design robustness.
- Simulation acceleration: AI-driven surrogate models provide near-real-time predictions [11], speeding up simulation processes.
- Generative AI for analog and RF design: Emerging tools like AnalogCoder [12] and PulseRF [13] enable faster design cycles and optimize performance.
- AI in early feedback loops: AI-powered tools can provide faster feedback during layout stages, real-time DRC (Design Rule Check) violation detection [14], supporting informed decision-making and improving productivity.

Showstoppers:

- Generalization: AI models often fail to adapt to new designs or nodes or designs that have different characteristics from the training data.
- Scalability: Limited scalability of current solutions for detailed routing.
- Data quality: Garbage-in, garbage-out issues in AI training data often limit the effectiveness of predictive models.
- Data availability: EDA and design data are limited and often not in the open domain. It is also difficult to extract data from existing design databases efficiently.
- Computational resource demands: AI-based methods can require significant computing power, making them inaccessible to smaller organizations.

- Integration challenges: Lack of seamless integration across EDA tools hinders the realization of end-to-end AI-driven workflows.

C. Opportunities for AI/EDA Collaboration

- Facilitating cross-community collaboration:
 - Open-source initiatives for developing shared datasets, tools, and benchmarks.
 - Community-wide competitions to foster collaboration and innovation in both AI and EDA communities.
 - Standardizing AI integration for interoperability across tools and workflows.
 - Joint research programs to address real-world challenges effectively.
 - Joint summer schools / bootcamps on AI + EDA to cultivate the next generation of workforce.
- Advanced modeling techniques: Collaborative research to improve multi-fidelity models for early-stage optimization, along with innovations in multi-modality AI.
- Open-source frameworks: Develop public benchmarks and datasets for AI-driven design, placement, and routing to enable reproducibility and innovation. Develop open-source frameworks to foster collaboration between AI and EDA communities, enabling the development, evolution, and contribution of integrated end-to-end solutions for AI+EDA in a collaborative manner.
- Hybrid AI approaches: Combine deep learning with traditional algorithmic approaches or heuristic-based methods to tackle hard optimization problems in EDA.
- Human-AI collaboration: A human-in-the-loop model enables seamless collaboration between AI tools and designers, allowing AI to handle repetitive tasks and complex simulations while leveraging human expertise and domain knowledge to guide, refine, and advance the development of AI solutions.

D. Future Directions and Recommendations to NSF

Future directions include, but are not limited to:

- Improve the generalization and scalability of AI models to effectively manage the complexities of modern chip design. Exploring the use of synthetic data and federated learning to address data scarcity and intellectual property concerns.
- Generative and gradient-based AI: Combining generative AI with gradient-based methods shows promise in optimizing physical design while balancing performance, power, and area (PPA).

- Agentic systems to automate physical design tasks, including design flow generation and physical design de-bug, etc.
- Multi-modal models to understand diverse data modality in the physical design, such as images, netlist/graph, RTL/text.
- Faster design cycles: Focus on streamlining manual tasks, enabling faster iteration cycles, improving overall throughput.
- End-to-end automation: Focus on creating systems capable of automating chip design from architecture specification to GDSII.
- Cross-layer co-optimization: Optimize across multiple design stages, including logic synthesis, placement, and routing.
- Human-in-the-loop systems to guide AI-driven design processes. Enhanced explainability for building trust and debugging AI decisions.
- Multi-fidelity models to bridge early-stage design and late-stage validation.
- AI-enhanced DFM: Incorporate ML-based surrogate models to rapidly predict and optimize manufacturability and yield during the design phase.
- Parallel processing: Leverage GPU/TPU-accelerated frameworks to significantly reduce turnaround times for large-scale optimization problems.
- Multimodal generative AI: Expand the use of multi-modal transformers to leverage all the different modalities of data available in the DFM.
- 3D IC integration: Explore AI-driven design and optimization techniques for 3D ICs, addressing challenges in vertical interconnect planning, thermal management, and multi-die co-design.
- Data infrastructure for EDA: open-source and commercial data infrastructures to extract EDA data efficiently from open-source and commercial EDA platforms.
- Benchmarks for AI on EDA: diverse sets of benchmarks for various sets of EDA problems, such as placement, optimization, routing, CTS, as well as analog/RF optimization and layout problems.

Based on the discussions, we propose the following

recommendations to NSF:

- Support foundational AI research for EDA: Focus on AI techniques tailored for EDA with explainability, scalability, generalization, and robustness.
- Encourage cross-disciplinary collaboration: Facilitate partnerships through workshops, competitions, and funding programs.
- Encourage open-source collaboration: Create incentives for industry and academia to develop and share datasets, benchmarks, and AI models.

- Promote scalable compute infrastructure: Provide grants for access to high-performance computing environments needed for training large AI models.
- Drive innovation in cross-layer Design Technology Co-Optimization (DTCO): Establish specific programs to fund digital-twin research and AI-driven DTCO tools.
- Invest in workforce development: Enhance education programs to train professionals in the intersection of AI and EDA to foster a skilled workforce.
- Support benchmark and dataset efforts: solicit and support benchmarks and datasets for AI for EDA tasks; propose grand challenges and contests on AI for EDA.

IV. AI for High-Level and Logic-Level Synthesis

A. Problem Description and Challenges

Problem Description: High-Level Synthesis (HLS) and Logic-Level Synthesis are pivotal stages in hardware design, translating high-level behavioral descriptions into gate-level specifications. However, these processes are traditionally manual, error-prone, and immensely time-consuming, requiring deep domain expertise that is in short supply. AI, particularly techniques like Machine Learning (ML) and Large Language Models (LLMs), offers the potential to automate and optimize these synthesis stages, which could significantly speed up the design process while also enhancing the performance, power, and area (PPA) efficiency of the resulting circuits. This session and topic focus aims to delve into the integration of AI in HLS and logic-level synthesis, exploring how AI can address the inherent challenges in these stages. Specific focus areas will include:

- Automating design decisions: AI, particularly through the use of machine learning models like GNNs and LLMs, can automate the insertion of pragmas and optimize other design decisions that traditionally require deep domain expertise.
- Predicting performance outcomes: AI models can predict chip performance early in the design phase, allowing for rapid iterative improvements without the need for full synthesis runs.
- Enhancing design space exploration: AI can significantly expand and efficiently explore the design space to identify optimal design configurations, far beyond what is feasible with traditional methods.

Challenges: The integration of AI into high-level and logic-level synthesis (HLS and LLS) is facing several critical challenges that stem from both workforce disparities and technical complexities:

- Labor disparity in hardware design: The significant gap between the number of software developers

(approximately 1,795,300) and hardware designers (78,100) highlights a labor crisis in hardware design. This disparity is problematic as it limits the number of skilled personnel available to meet the increasing demand for specialized hardware. To counter this, research has been directed towards empowering software developers to participate in chip design, aiming to democratize and accelerate the hardware design process.

- **Data challenges:** AI-driven approaches in HLS and LLS heavily rely on high-quality, labeled data, which is scarce, proprietary, sensitive, and costly to produce. The challenge lies in developing data-sharing mechanisms that protect the proprietary aspects of designs while providing sufficient data to train effective AI models. This balance is crucial for advancing AI capabilities in chip design without compromising intellectual property.
- **Integration with existing EDA tools:** Integrating AI tools into the existing complex and often outdated EDA software stacks presents another hurdle. AI models must seamlessly interact with these tools, requiring extensive engineering efforts and a profound understanding of both software and hardware domains. This integration is essential for practical application, leveraging AI innovations effectively in the synthesis process.
- **Trust and reliability:** There is a fundamental need for trust in AI-generated designs, which must adhere to stringent industry standards. Building this trust involves developing comprehensive validation and verification processes that are both thorough and efficient. However, these processes can be resource-intensive and challenging to establish, posing a significant barrier to the routine use of AI in critical design tasks.

These challenges underscore the complexities of applying AI to revolutionize HLS and LLS. Addressing these issues requires not only technological innovations but also strategic collaborations across the industry to enhance data availability, tool integration, and trust in AI-generated outputs.

B. Summary of Existing Efforts and Showstoppers

High-Level Synthesis. Currently, there are two types of AI-assisted efforts in design creation. One starts with a high-level specification, such as C/C++ or even Python, and gets refined all the way down to RTL code with the help of high-level synthesis (HLS) coupled with deep learning optimization (HLS+DL). The other effort is more ambitious, starting with natural languages and generating RTL code directly (NL2RTL).

HLS+DL. For the class of HLS+DL approaches in the first category, an early work is GNN-DSE [15], which represents the input C/C++ programs as a control and data-flow graph (CDFG), and uses graph neural networks to build a predictive model so that circuit performance can be predicted in milliseconds without running the HLS tool. GNN-DSE is trained using a database of HLS designs generated by an expert-like system called AutoDSE [16]. This model is enhanced to HARP [17], which adds hierarchical modeling to deal with the long-dependency issues in the input program. HARP also models pragmas as function transformations. With the rapid advance in LLM, a subsequent work, ProgSG [18], combines GNN and LLM with cross-modality attention.

One serious challenge of any of these predictive models is to track the changes of the toolchain (e.g., the HLS tool) they plan to model. This problem is addressed by Active-CEM [19], which carries out a task transfer learning scheme that leverages a model-based explorer designed to adapt efficiently to changes in toolchains. This approach optimizes sample efficiency by identifying high-quality design configurations under a new toolchain without requiring extensive re-evaluation. It further refines the methodology by incorporating toolchain-invariant modeling.

Another major challenge arises when the trained model is applied to new designs, as the substantial domain shift often results in unsatisfactory performance. One attempt to address this problem is to use a hierarchical MoE [20], which proposes a more domain-generalizable model structure. Different expert networks can learn to deal with different regions in the representation space, and they can utilize similar patterns between the old kernels and new kernels. The MoE can be applied at the basic block level, loop/function level, and the graph level, with the assumption that even for a new kernel, it will share common basic blocks and loop structures with some training examples, although the composition of these basic structures can be different. This approach showed some promising results for adapting to new domains. Nevertheless, more effective and robust methods are needed to transfer learning to new designs.

So far, these HLS+DL methods focus on pragma insertion, which is very important in defining the microarchitecture of the chip, and rely on the underlying Merlin compiler [21] to carry out limited program transformation, such as loop strip mining and memory access coalescing. However, more substantial transformation of the original program, e.g., with loop ordering and distribution, may achieve considerably better results, as demonstrated in several non-ML-based approaches, including ScaleHLS [22], HIDA [23], Allo [24], StreamHLS [25],

and Sisyphus [26]. However, most of these approaches are limited to affine programs, so that some analytical models can be derived for performance prediction. It will be interesting to incorporate these methods and expand the design space of the existing HLS+DL methods to include program transformation for non-affine programs.

NL2RTL. For the NL2RTL efforts in the second category, which begin with natural language inputs to generate RTL code, early explorations like GPT4AIGChip [27] demonstrate the feasibility of this approach. GPT4AIGChip processes natural language specifications that define target design specifications and resource constraints, such as latency and DSPs, to produce RTL designs according to the provided demonstration examples of AI accelerators. Initially validated in the context of CNN accelerators, this method illustrates how advanced language models can potentially simplify and automate complex design tasks.

Motivated by the effectiveness of merely providing a few demonstration examples in GPT4AIGChip, MG-Verilog [28] introduces an approach to enhancing LLM-assisted Verilog generation by providing a dataset of 11k that includes multi-grained, detailed descriptions of hardware functions alongside their corresponding Verilog code. This dataset aims to improve the training and operational efficacy of language models in hardware design by offering diverse levels of description detail, from high-level summaries to intricate, line-by-line annotations.

LLM4SVA [under review] presented a further development in this direction, targeting the automated generation of SystemVerilog Assertions (SVAs) from high-level natural language descriptions. This framework aims to bridge the verification gap that arises with NL2RTL approaches by providing an LLM-based method to ensure that the RTL code/design adheres to its specifications, thereby enhancing reliability and reducing manual verification efforts.

The NL2RTL direction still faces significant challenges, particularly in terms of accuracy and the faithful translation of functional requirements into hardware designs. To address these concerns, continuous advancements in language model training methodologies and hardware architecture-specific adaptations are being explored. The integration of domain-specific knowledge into the training process of LLMs is crucial for achieving higher precision and functional correctness in generated RTL code.

Moreover, the verification of designs generated through NL2RTL approaches remains a critical hurdle. It requires innovative solutions to ensure that the designs not only meet performance metrics but also

adhere strictly to safety and regulatory standards. The development of integrated tools that can automatically generate and verify RTL from natural language will be key to the maturation and acceptance of NL2RTL technologies in mainstream hardware design workflows.

Overall, while still in the experimental phase, the NL2RTL effort holds the promise of significantly streamlining the design process, making it faster and more accessible to designers without deep expertise in traditional hardware description languages. These developments, paired with advancements in AI and machine learning, are poised to redefine the landscape of digital design in the coming years.

Having high-quality training data is important for the success of the approaches in both categories. The Hlsyn benchmarks [29] provided 80,000 HLS programs with performance and resource utilization data, which was very helpful for several subsequent research (see [18], [19], [20]). The Chrysalis dataset [30] (symbolizing a phase in the life cycle where a “bug” transforms into a “butterfly”) includes an extensive collection of over 1,500 HLS designs, annotated with 19 distinct bug types through LLM-based bug injection techniques. A contest was organized at ICCAD 2024 to collect benchmarks for NL2RTL efforts [31]. More efforts are needed in this direction so that we can reach the training data size of ImageNet scale.

Logic-Level Synthesis. On the logic-level synthesis front, the following ML-driven works have been studied, showcasing promising strides in addressing challenges such as design optimization, resource usage estimation, and synthesis automation. Leveraging ML techniques like deep neural networks (DNNs), reinforcement learning (RL), and symbolic reasoning, these works have advanced the quality of results (QoR) and efficiency in logic synthesis of digital chip design.

- Accurate estimation of design metrics with ML: High-Level Synthesis (HLS) often encounters discrepancies between estimated and post-implementation metrics. To address this, Pyramid [32] introduces an ensemble ML model trained on diverse benchmarks that predicts timing and resource usage with over 95% accuracy. Thus, it reduces the need for exhaustive implementation and enables early assessment of design trade-offs.
- Dataset generation for ML tasks in logic synthesis: Datasets are foundational for ML models in logic synthesis. OpenLS-DGF [33] generates over 966,000 Boolean circuits in Verilog and GraphML formats, supporting various ML tasks like circuit classification and QoR prediction, advancing ML applications in logic synthesis.

- Automation through reinforcement learning: The exploration of vast design optimization spaces benefits significantly from RL-based approaches:
 - DRILLS [34] applies the Advantage Actor-Critic (A2C) algorithm to navigate optimization spaces autonomously. It achieves a 13% average QoR improvement on EPFL benchmarks by minimizing area under timing constraints.
 - AISYN [35] extends this concept by overcoming local minima in classical synthesis methods. Leveraging RL, AISYN optimizes delay, area, and power, achieving up to a 69.3% reduction in cell area compared to traditional methods. These works highlight RL's potential to outperform human-driven optimization processes.
- Hybrid approaches to logic synthesis: LOracle [36] integrates traditional synthesis techniques with AI by combining And-Inverter Graph (AIG) and Majority-Inverter Graph (MIG) optimizers. Utilizing DNNs to classify circuit partitions, it applies the most suitable optimizer, achieving a 6.87% improvement in area-delay product for 7 nm ASIC designs. This hybrid framework bridges the gap between automation and human expertise.
- Logic minimization with synergistic learning: The INVICTUS [37] framework addresses logic minimization by automating optimization sequence generation through model-based offline RL. By combining RL and search methods, INVICTUS synthesizes Boolean circuits, achieving up to a 30% improvement in area-delay product (ADP) and a 6.3× reduction in runtime. This scalability positions it as a powerful solution for various circuit benchmarks.
- Symbolic reasoning for functional understanding: Gamora [38] applies symbolic reasoning at scale, using graph neural networks (GNNs) and GPU acceleration to extract high-level functional blocks from gate-level netlists. Supporting large-scale Boolean networks, Gamora enhances the analysis and understanding of complex designs, opening avenues for advanced functional insights.

These ML-driven advancements highlight the transformative potential of AI in logic synthesis, enabling more efficient, accurate, and automated workflows. From precise metric estimation and adaptive dataset generation to RL-based optimization and hybrid frameworks, these works represent significant progress. However, challenges remain in generalizing across diverse designs, bridging theory with industrial adoption, and scaling for emerging technologies. Part of the challenge stems from the lack of ML-friendly representations of EDA data, as well as the inherent complexity of

integrating logical, geometric, and numeric reasoning into ML.

C. Opportunities for AI/EDA Collaboration

Opportunities for AI Community:

- Logic synthesis and high-level synthesis (HLS) provide unique new problems to the AI community. AI researchers prefer problems with clear inputs and outputs, which can be fairly evaluated and benchmarked. These problems have common challenges seen by other AI applications, such as multi-modality fusion, domain transfer, task transfer, and label scarcity. These problems have also raised unique challenges, such as delayed reward, expensive to run evaluation, and an extremely large search space.
- AI for optimization can find a new arena in this area.
- AI for simulation would be an essential technique to help accelerate the process of collecting (early) feedback for designs.
- How to effectively leverage pre-trained foundation models to turn natural language into specs and then Verilog will be interesting.

Opportunities for EDA Community:

- AI tools can significantly accelerate the pipelines and simplify the workflow involved in EDA.
- AI for simulation can be used to generate large-scale synthetic datasets.
- GenAIs and black-box optimization can be used to propose high-quality design points without the necessity to brute-force search.
- LLM-based agents can work with human experts.

How to Facilitate Cross-Community Collaboration:

- Build more benchmark datasets and set up leaderboards.
- Advertise more in the AI community and host contests. A good venue could be the NeurIPS contest track. The AI community uses social media significantly.
- Continue AI4EDA workshops in ML conferences.
- Introduce AI topics into EDA courses and introduce EDA problems into AI courses.

D. Future Directions and Recommendations to NSF

Future Directions on EDA:

- Agentic-EDA: Agents are enabling an exciting new paradigm in the evolution of AI systems—systems that build intelligence at inference/real-time vs training time only, and are shifting the system behavior from feedforward systems (prompt-LLM-output) to feedback systems (Think/Plan-Act with Tools—Observe/Reflect-Loop -Re-Think/

Plan-Act with Tools-Observe/Reflect-...). This offers the opportunity of a significant productivity boost for the chip designers as the level of automation can move from a single sub-task to a complex end-to-end task, like fixing a bug/timing/area/noise/power issue in a design with agentic workflows. A multi-agent system that combines various sub-agents specializing in different aspects of EDA with human-in-the-loop is the future of agentic-EDA.

- Carefully engineered AI/ML tools: To help chip designs, the following directions are promising: combining multiple ML models, such as GNN and LLM, for more comprehensive representations of the circuit; capturing design hierarchy and pragma transformations and multi-modality attention; task transfer learning with active learning and domain transfer learning with MoEs, etc. In addition, ML may be useful when (1) an IC is implemented many times after small changes (then we have a large dataset for training); (2) faster prototyping of combinatorial solvers; (3) optimization achieved through Reinforcement Learning, even when the optimization objective is not available in a closed form.
- Extracting and learning of information: Streamline the process of extracting/learning of information from existing designs and optimization runs, and apply such knowledge/expertise to speed up and enhance future designs. This strategy can be applied to virtually any aspect of the EDA design and verification space, including Logic and High-Level Synthesis. Examples include: (1) Design to design: learn from existing designs to predict PPA for new designs; (2) Run to run: serve as congestion/timing/area predictor during optimization; (3) Learning and predicting the desirable Technology / Library configurations.
- Predicting key performance metrics: New ML models trained to predict key performance metrics from high-level code or intermediate representations for circuits, reducing dependency on slow simulation/implementation, which can help DSE and identify high-quality design points. This can be high-level architecture decisions at HLS and suitable optimization at the logic level.

Other Promising Directions:

- Prioritize optimization moves (e.g., retiming, logic restructuring, sizing, placement), based on learned performance trade-offs.
- Generate new datapath elements optimized for specific metrics, instead of relying on library templates.

- Enhance “Lint” verification and predict problematic code, possible congestion, and timing problems based on learned experience from previous designs.
- Assist in interpreting implementation error messages and suggesting fixes.
- Focus on predictability, stability, repeatability, and generalizability (over-fitting results to a particular data set might limit QoR).

Break the Data Barrier in EDA

- A framework to develop a truly open EDA LLM with a taxonomy, synthetic data generation, and iterative large-scale alignment/tuning approach (e.g., IBM’s open community and open source InstructLab approach) that enables LLM contributions of knowledge and skills from the community. This will help us overcome the data bottleneck in EDA, which has hampered the progress of AI in the Chip Design Automation domain.
- Large-scale RTL code generation: Use high-performance HLS flows to generate a large amount of high-quality RTL code, e.g., leveraging the HIDA flow [23] and Allo flow [24].
- Utilizing low-quality data for training: Semiconductor design teams create huge amounts of “low quality” data on a daily basis: implementation tools log files and reports; tool settings and run options; PPA info at various implementation stages, etc. It will be helpful to create a process to bring this data to a level that is useful for training purposes.

Recommendations to NSF:

- Promote benchmarking data and tasks.
- Promote open frameworks and platforms.
- Encourage new AI tools, such as LLM-based agents.
- Encourage novel multi-modal AI models that are generalizable, stable, and interpretable.
- Verification should be included in the workflow.

V. AI for Test and Verification

A. Problem Description and Challenges

Test and verification encompass a very broad topic covering a wide range of robustness challenges: (a) design bugs introduced during chip design; (b) manufacturing defects and hardware reliability failures; (c) security vulnerabilities of chips; (d) fragility and security vulnerabilities of AI-based software systems.

Hardware design verification of chips is a major, perhaps the most significant, design challenge today. Pre-silicon verification is used to detect logic design bugs before chips are manufactured. Several industrial

reports highlight significant challenges of existing pre-silicon verification practices (e.g., 80% of design effort may be dedicated to verification, first silicon success lowest since 2004 [39]). The rise of hardware accelerators and multi-chiplet systems makes verification even more challenging. Many design variants and their combinations must now be verified thoroughly and quickly (much quicker and with much more nimble design teams, unlike processors). Major verification challenges include: (a) tremendous efforts in crafting design-specific assertions, tests, properties, and full functional specifications, and (b) critical bug escapes despite “100% coverage” highlighting the inadequacy of existing approaches [40]. These challenges get further magnified with AI-generated hardware designs—since AI can generate designs quickly, the relative verification effort increases commensurately.

Bugs that escape pre-silicon verification are detected during post-silicon validation (chips are tested in actual system environments to detect and fix bugs). In addition to logic design bugs, post-silicon validation also targets electrical bugs that are caused by subtle interactions between a design and its electrical state (such as crosstalk, power-supply noise, thermal effects). Post-silicon validation is extremely difficult because the internal signals of a chip are no longer available for controllability and observability, unlike pre-silicon verification.

Beyond design bugs, manufacturing defects are also a major concern. Software engineers typically assume that commodity compute chip hardware—e.g., CPU, GPU, ML accelerator—always works correctly during its lifetime. Large-scale distributed systems in data centers often assume that hardware errors, if any, immediately cause software execution to stop via crashes and hangs, for example, the fail-stop failure assumption. Unfortunately, none of these assumptions holds today. Too many defective compute chips—i.e., with manufacturing defects—are escaping today’s manufacturing tests—at least an order of magnitude more than industrial targets across all compute chip types in data centers. Silent data corruptions (SDCs) caused by test escapes, when left unaddressed, pose a major threat to reliable computing [41].

Reliability failures, largely benign in the past—radiation-induced soft errors, early-life failures, circuit aging—cause unexpected data corruption and expensive system downtimes with severe consequences [42]. Business-as-usual manufacturing-time testing, large speed/voltage margins, or expensive redundancy for fault-tolerance aren’t scalable or cost-effective anymore.

The advent of AI-based optimization has dramatically reshaped the landscape of hardware security, presenting unprecedented challenges. The attack surface

has expanded to an alarming degree, encompassing not only traditional vulnerabilities within the hardware itself but also novel avenues introduced by the AI integration. Each stage of the design and optimization process, now heavily influenced by AI algorithms, necessitates a rigorously defined formal attack model to effectively identify and mitigate potential threats. This is particularly crucial as the very intelligence intended to optimize design and performance simultaneously introduces new vectors for sophisticated attacks, demanding a proactive and comprehensive re-evaluation of security paradigms. Similarly, the robustness of software systems, which use AI at their core, is becoming highly challenging as well. This is featured by the deep system stack from chips to code, as well as the heterogeneous compute involving non-determinism and complex interactions across hardware platforms, runtime environments, and software ecosystems.

B. Summary of Existing Efforts and Showstoppers

LLMs have been explored for hardware design using RTL (such as Verilog) from natural language prompts. Examples of such efforts include [27], [43], [44], [45], [46], [47], [48], [49]. However, these approaches have not yet delivered the expected productivity boost because they suffer from the fundamental design verification bottleneck discussed in Section V-A—which gets even more severe with hallucinations inherent to today’s AI models and ambiguity inherent to natural language prompts. As a result, frontier models produce RTL designs that are riddled with a prohibitively large number of bugs for complex designs, which is one of the primary reasons that those efforts have not yet delivered the expected productivity boost.

While HLS techniques, discussed in Section IV, are promising, HLS also suffers from the design verification bottleneck challenges especially when domain-specific ASICs are targeted. In addition, bugs introduced in HLS pragmas, introduced by human designers or by AI, can be extremely tricky to detect.

AI techniques for design verification and bug fixing (e.g., LLM- or RL-based, such as LLM4SVA discussed in Section IV and others) are in their infancy today—especially for “difficult” corner-case bugs that get detected by specific input sequences. Existing verification techniques are already highly challenging, and critical bugs escape existing verification flows that are highly time-consuming and resource-intensive in terms of human expertise. Thus, targeting AI techniques that match the quality of existing verification flows isn’t sufficient to ensure design quality and overall design productivity moving forward. Even when bugs are successfully detected, fixing those bugs must also be automated for

AI-driven design flows for overall design productivity. Very little progress has been made in the above areas. Needless to say, high-quality training data is crucial for success (e.g., see the discussion on the Chrysalis dataset and corresponding bug injection techniques in Section IV).

A combination of AI-driven design techniques and recent advances in a new formal verification approach (G-QED [40]) provides a promising path as presented in [50] and explained in Section V-C. This approach exploits the unique individual strengths of AI and formal verification in synergistic ways for large productivity and design quality benefits while bypassing the weaknesses of AI approaches for design verification and bug fixing.

Security for hardware and embedded software in the age of AI presents a distinct set of challenges and advantages. Due to the typically lower level of abstraction and the imperative for real-time performance, embedded software often benefits from having no hidden states, making its behavior more predictable and auditable. This inherent transparency, coupled with highly constrained and well-defined use cases, contributes to a more robust security posture compared to general-purpose software. Furthermore, the significantly smaller codebases characteristic of embedded systems reduce the overall attack surface. However, the stringent real-time requirements introduce unique verification hurdles, as security measures must not compromise the system's deterministic and timely operation.

For AI-based optimization and hardware systems, ensuring robust security demands adherence to several critical criteria. Foremost is resilience against a spectrum of malicious inputs, including various poisoning and adversarial attacks that can subtly manipulate system behavior or compromise model integrity. Equally vital is the fidelity of AI model systems to the original design intent, preventing unintended deviations or exploitable backdoors introduced during training or deployment. Finally, meticulous attention must be paid to numerical precision and stability to prevent errors during training that could lead to convergence loss or outright divergence of the training process, ultimately undermining the reliability and security of the entire AI-driven hardware system.

C. Opportunities for AI/EDA Collaboration

Example 1:

One of the biggest AI/ML opportunities for chip design is in “creating” highly efficient and robust digital designs with drastic design productivity improvements—e.g., a few days vs. several months. Since verification is the biggest bottleneck in design today, design and

verification cannot be treated separately for overall productivity benefits. This creates a unique opportunity to combine AI/ML techniques synergistically with recent breakthroughs in design verification (such as G-QED [40] and its extensions), which includes non-AI/ML techniques, for an overall AI-driven approach (e.g., AI-Boosted Chip Design or ABCD approach in [50]). The key idea is to effectively combine these techniques by exploiting their strengths while avoiding their weaknesses.

To achieve this objective, several deeply technical questions must be answered. A few examples include: 1. How can AI/ML techniques be used to create designs with drastically fewer bugs to start with (before any design verification techniques are applied)? Today's approaches based on naive natural language prompts create designs riddled with bugs despite small design sizes and no advanced architecture features. 2. How can we verify complex AI/ML-generated designs with minimal human intervention using ideas in [40] and others? What verification guarantees can be provided, especially for classes of bugs introduced by AI/ML? 3. How can we feed verification results (e.g., counterexamples or bug traces) back to AI/ML for bug fixes? Can we rely on AI/ML-assisted debugging? 4. How do we ensure that bug fixes are correct and don't significantly degrade the performance and energy efficiency of resulting designs? Can AI/ML and non-AI/ML techniques collaborate on this aspect? 5. How do we ensure that AI/ML understands various architecture features “correctly” and uses them in “appropriate” ways when creating designs?

Example 2:

Little research literature exists on the effective use of AI for post-silicon validation and debug. Data-driven techniques have been explored for manufacturing testing, mainly from the perspective of reducing cost, e.g., manufacturing test times. However, as discussed in Section V-A, today's manufacturing testing practices face major challenges resulting in significant amounts of defective chips escaping into the field. There is an opportunity to explore the role of AI/ML in this context [41]. For example, one promising application is identifying potentially defective machines (i.e., machines containing defective chips) in data centers by analyzing system-level signals—uncovering “needle-in-a-haystack” patterns beyond existing rule-based methods. Another direction is using data-driven approaches for in-field testing and error detection—deciding when to schedule them, which hardware blocks to target, or what test content to apply—based on various hardware and software events and on-chip sensor data. Accurate diagnosis from system-level incorrect behaviors and insights obtained from well-designed test experiments can potentially generate high-quality training data for such AI/ML.

Finally, understanding how test escape-induced errors impact AI/ML workloads at the application level is critical for ensuring end-to-end robustness.

Example 3:

Another avenue for deep collaboration between the AI and EDA communities is the use of ML in solvers, provers, analyzers, and verifiers that are integral to EDA tools. Over the last decade, we have seen increasing integration of ML into EDA tools, resulting in dramatic improvement in performance and adaptability of these tools. This is best understood when we notice that solvers (e.g., SAT and SMT solvers), provers (e.g., first-order provers), analyzers (e.g., static analysis tools), and verifiers (e.g., bounded model checkers) are rule-based. Further, traditionally, these systems used ad-hoc heuristics at decision and choice points to optimally sequence, select, and initialize these rules. However, these ad-hoc rules can be replaced with ML-based heuristics, resulting in a novel class of neurosymbolic AI techniques [51], that are data-driven and solve the optimization problems associated with rule selection, sequencing, and initialization. Perhaps the best representatives of this approach are Maple* series of SAT solvers that have had a dramatic improvement in the SAT and SMT solver community [52], and the benefits of which have translated into far more performant EDA tools today.

Once again, the key insight of this line of work is that there is great benefit in combining ML and verification tools in a symbiotic corrective feedback loop (e.g., reinforcement learning with symbolic feedback [53]) where the verifier provides corrective feedback to the generative ML and the ML generates formal objects (and data) for the verifier/prover to check [51].

D. Future Directions and Recommendations to NSF

- A neurosymbolic symbiotic loop between solvers/provers/verifiers and AI: NSF should promote research in neurosymbolic AI techniques that enable the development of ML techniques to improve solvers/provers/verifiers, and in the reverse direction, use solvers/provers/verifiers and domain knowledge to improve the trustworthiness of ML.
- A neurosymbolic symbiotic loop between synthesizers and AI: AI tools, such as LLMs, are surprisingly good at generating code, proofs, and other formal objects. Having said that, with corrective feedback from provers and verifiers during training and fine-tuning (e.g., RLSF), the quality of their output can be dramatically improved. Hence, NSF should focus on combining synthesizers in corrective feedback loops with AI systems.
- Neurosymbolic AI tools for HLS and RTL synthesis/verification and bug-fixing: NSF should also

promote the use of provers/verifiers as providing symbolic feedback to AI-driven synthesis during inference (in-context symbolic feedback).

- Neurosymbolic AI tools post-silicon, debug, reliability: NSF should promote similar neurosymbolic AI tools for post-silicon, debug, and reliability.
- AI software/embedded systems and security—AI systems introduce new attack surfaces that need to be addressed.

Acknowledgment

This work was supported in part by NSF under Grant 2436036, in part by Google, in part by AMD, and in part by the Institute for Learning-enabled Optimization at Scale (TILOS). The authors would like to thank the speakers and panelists for their insightful contributions, including Deming Chen, Jeff Dean, Vijay Ganesh, Aditya Grover, Farinaz Koushanfar, Koen Lampaert, Yingyan Lin, Yong Liu, Igor Markov, Subhasish Mitra, Bryan Perozzi, Pranay Prakash, Ruchir Puri, Mark Ren, Shobha Vasudevan, Yusu Wang, and Lingming Zhang. They also thank the workshop organizers for their leadership and coordination, including Jason Cong, Sergio Guadarrama, Stefanie Jegelka, David Z. Pan, and Yizhou Sun. They gratefully acknowledge the support of the NSF program directors, including Raj Acharya, Sankar Basu, and Sharon Hu. Finally, they thank the student volunteers who assisted with the organization, including Zijian Ding, Weikai Li, Zongyue Qin, and Derek Xu.



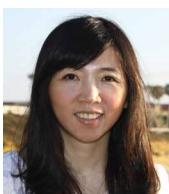
Deming Chen (Fellow, IEEE) received the Ph.D. degree in computer science from UCLA in 2005. He is a Abel Bliss Professor at the Grainger College of Engineering, University of Illinois Urbana-Champaign. He has published more than 300 research papers, received ten Best Paper Awards, two ACM/SIGDA TCFPGA Hall-of-Fame Paper Awards, five Best Poster Awards, and delivered more than 170 invited talks, including over 20 keynote and distinguished lectures. His research interests include machine learning and AI, system-level design methodologies, hybrid cloud systems, security and confidential computing, and reconfigurable and heterogeneous computing. He is an ACM Fellow and previously served as the Editor-in-Chief for *ACM Transactions on Reconfigurable Technology and Systems* (TRETS). Under his leadership, the impact factor of ACM TRETS has increased by 3.8 times. He serves as the Illinois Director for the IBM-Illinois Discovery Accelerator Institute and the Director of the AMD Center of Excellence. Additionally, he has been involved in several startup companies, including AutoESL and Inspirit IoT.



Vijay Ganesh received the Ph.D. degree in computer science from Stanford University in 2007. He is a Professor of computer science and an Associate Director of the Institute for Data Engineering and Science (IDEaS), Georgia Institute of Technology. Additionally, he is the Co-Founder and the Director of the AI for Math Center, Georgia Tech, and the Co-Founder of the Centre for Mathematical AI, Fields Institute. Prior to joining Georgia Tech, in 2023, He was a Professor at the University of Waterloo from 2012 to 2023, the Co-Director of the Waterloo AI Institute from 2021 to 2023, and a Research Scientist at MIT from 2007 to 2012. His primary research area is the theory and practice of SAT/SMT solvers, combinations of machine learning and automated reasoning, and their application in neurosymbolic AI, software engineering, security, mathematics, and physics. He has led the development of many SAT/SMT solvers, most notably, STP, Z3str family of string solvers, Z3-alpha, MapleSAT, AlphaMapleSAT, and MathCheck. He also leads the development of several neurosymbolic AI tools and aimed at mathematics, physics, and software engineering. On the theoretical side, he works on topics in mathematical logic and proof complexity. For his research, he has won over 35 awards, honors, and medals, including the ACM Impact Paper Award at ISSTA 2019, the ACM Test of Time Award at CCS 2016, and the Ten-Year Most Influential Paper citation at DATE 2008.



Weikai Li received the bachelor's degree in computer science and technology from Tsinghua University in 2023. He is currently pursuing the Ph.D. degree with the Computer Science Department, UCLA, under the supervision of Prof. Yizhou Sun. His research focuses on knowledge-enhanced AI for chip design, improving the capability and generalizability of graph neural networks (GNNs) and large language models (LLMs) for chip design.



Yingyan (Celine) Lin is an Associate Professor at the School of Computer Science and the Co-Director of the Center for Advancing Responsible Computing, Georgia Institute of Technology. She leads the Efficient and Intelligent Computing (EIC) Laboratory, focusing on efficient AI solutions, with an emphasis on cross-layer algorithm-hardware co-design and agentic AI techniques for hardware design. Her group's research has

earned various recognitions, including First Place at the ACM SIGDA University Demonstration at DAC 2022, First Place at the ACM/IEEE TinyML Design Contest at ICCAD 2022, and selection as the IEEE Micro Top Pick of 2023 for "the most significant research papers in computer architecture based on novelty and potential for long-term impact." Additionally, their work has been spotlighted at ICLR in 2020, 2021, and 2025, and NeurIPS 2025, presented as an oral paper at ECCV 2024, and received the Best Paper Award at MICRO 2024. She is the PC Co-Chair for both MLSys 2025 and the IEEE MICRO Top Picks 2026.



Yong Liu received the M.S. degree in computer science from Fudan University. He is the Senior Group Director at Cadence Design Systems, where he leads research and development teams on large-scale data analytics, artificial intelligence, LLM infrastructure, and applications, including JedAI and ChipGPT. He is an Experienced Architect in electronic design automation (EDA), AI hardware-software co-design, and AI-driven data analytics. Prior to joining Cadence, he contributed to AI chip development and formal verification tooling across multiple technology companies. He is the Technical Program Committee (TPC) Co-Chair of IEEE International Conference on LLM-Aided Design.



Subhasish Mitra (Fellow, IEEE) holds the William E. Ayer Endowed Chair Professorship at the EECS Department, Stanford University. He directs the Stanford Robust Systems Group, serves on the leadership team of the Microelectronics Commons AI Hardware Hub funded by the US CHIPS and Science Act, leads the Computation Focus Area of the Stanford SystemX Alliance, and is the Associate Chair (Faculty Affairs) of Computer Science. His research ranges across robust computing, nanosystems, electronic design automation (EDA), and neurosciences. He is a fellow of ACM. His honors include the Harry H. Goode Memorial Award (by IEEE Computer Society), Newton Technical Impact Award in EDA (test-of-time honor by ACM SIGDA and IEEE CEDA), the University Researcher Award (by Semiconductor Industry Association and Semiconductor Research Corporation), the EDAA Achievement Award (by European Design and Automation Association), the Intel Achievement Award (Intel's highest honor), and the Distinguished Alumnus Award from the Indian Institute of Technology, Kharagpur.



David Z. Pan (Fellow, IEEE) received the B.S. degree from Peking University and the M.S. and Ph.D. degrees from the University of California at Los Angeles. He is a Professor and a holder of the Silicon Laboratories Endowed Chair at the ECE Department, UT Austin. He has published over 500 refereed papers at top EDA/chips/architecture and machine learning venues. His research interests are mainly focused on electronic design automation (EDA), including how to leverage AI/ML for EDA in physical design and design for manufacturing, as well as synergistic AI and IC co-optimizations. He is a fellow of ACM and SPIE. He has served as the Program Chair for DAC in 2024 and ICCAD in 2018, and the two flagship conferences in EDA. He has received many awards, including the SRC Technical Excellence Award, the DAC Top 10 Author in Fifth Decade, the DAC Prolific Author Award, the ASP-DAC Frequently Cited Author Award, the 21 Best Paper Awards, and over 20 additional Best Paper candidates/finalists, and a number of international CAD contest awards, among others.



Ruchir Puri (Fellow, IEEE) is the Chief Scientist of IBM Research, an IBM Fellow, and the Vice-President of IBM Corporate Technology. He led IBM Watson as the CTO and the Chief Architect from 2016 to 2019 and has held various technical, research, and engineering leadership roles across IBM's AI and Research businesses. He has been an ACM Distinguished Speaker, an IEEE Distinguished Lecturer, and was awarded the 2014 Asian American Engineer of the Year. He has been an Adjunct Professor at Columbia University, NY, and a Visiting Scientist at Stanford University, CA. He was honored with the John Von-Neumann Chair at the Institute of Discrete Mathematics at Bonn University, Germany. He is an Inventor of over 75 U.S. patents and has authored over 150 scientific publications. He is the Founder of the IEEE LLM-Aided Design Conference. He was a recipient of the Distinguished Alumnus Award from Indian Institute of Technology (IIT), Kanpur.



Jason Cong (Fellow, IEEE) is the Volgenau Chair for Engineering Excellence Professor at the Computer Science Department, UCLA, with joint appointment from the Electrical and Computer Engineering Department. He is the Director of the Center for Domain-Specific Computing (CDSC) and the Director of the VLSI Architecture, Synthesis, and Technology (VAST) Laborato-

ry. His research interests include novel architectures and compilation for customizable computing, synthesis of VLSI circuits and systems, and quantum computing. He is a member of the National Academy of Engineering, the American Academy of Arts and Sciences, and a fellow of ACM and the National Academy of Inventors. He was a recipient of the SIA University Research Award and the EDAA Achievement Award. He was honored for the IEEE Robert N. Noyce Medal "for fundamental contributions to electronic design automation and FPGA design methods" in 2022, the Phil Kaufman Award, the highest recognition in EDA, in 2024, and the ACM Chuck Thacker Breakthrough Award in 2025 "for fundamental contributions to the design and automation of field-programmable systems and customizable computing."



Yizhou Sun (Senior Member, IEEE) received the Ph.D. degree in computer science from the University of Illinois at Urbana-Champaign in 2012. She is a Professor at the Department of Computer Science, UCLA, and an Amazon Scholar. She is a Pioneer Researcher in mining heterogeneous information networks, with a recent focus on deep learning on graphs, AI for chip design, AI for science, and neuro-symbolic reasoning. She has over 200 publications in books, journals, and major conferences. Tutorials on her research have been given in many premier conferences. She was a ACM Distinguished Member. She was a recipient of multiple Best Paper Awards, the ACM SIGKDD Doctoral Dissertation Award, the Yahoo Academic Career Enhancement (ACE) Award, the NSF CAREER Award, the CS@ILLINOIS Distinguished Educator Award, the Amazon Research Awards (twice), the Okawa Foundation Research Award, the VLDB Test of Time Award, the WSDM Test of Time Award, the IEEE AI's 10 to Watch, and the SDM/IBM Faculty Award. She is the General Co-Chair of SIGKDD 2023, the PC Co-Chair of ICLR 2024, and the PC Co-Chair of SIGKDD 2025.

References

- [1] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.
- [2] A. F. Budak et al., "DNN-opt: An RL inspired optimization for analog circuit sizing using deep neural networks," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 1219–1224.
- [3] Z. Guo et al., "A timing engine inspired graph neural network model for pre-routing slack prediction," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 1207–1212.
- [4] C.-C. Chang et al., "DRC-Coder: Automated DRC checker code generation using 11m autonomous agent," in *Proc. Int. Symp. Phys. Design (ISPD)*. Austin, TX, USA: ACM, Mar. 2025, pp. 143–151.
- [5] Y.-C. Lu et al., "DREAM-GAN: Advancing dreamplace towards commercial-quality using generative adversarial learning," in *Proc. Int. Symp. Phys. Design*, 2023, pp. 141–148.

- [6] Y. Lin et al., "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," in *Proc. 56th Annu. Design Autom. Conf.*, Jun. 2019, pp. 1–6.
- [7] Y.-C. Lu et al., "INSTA: An ultra-fast, differentiable, statistical static timing analysis engine for industrial physical design applications," in *Proc. 62nd ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2025, pp. 1–7.
- [8] Z. Xiong, R. S. Rajarathnam, and D. Z. Pan, "A data-driven, congestion-aware and open-source timing-driven FPGA placer accelerated by GPUs," in *Proc. IEEE 32nd Annu. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, May 2024, pp. 115–125.
- [9] D. Ding et al., "Machine learning based lithographic hotspot detection with critical-feature extraction and classification," in *Proc. IEEE Int. Conf. IC Design Technol.*, May 2009, pp. 219–222.
- [10] W. Ye et al., "LithoGAN: End-to-end lithography modeling with generative adversarial networks," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, 2019, pp. 1–6.
- [11] D. Boning et al., "NeurOLight: A physics-agnostic neural operator enabling parametric photonic device simulation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 14623–14636.
- [12] Y. Lai et al., "AnalogCoder: Analog circuit design via training-free code generation," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 1, pp. 379–387.
- [13] H. Chae et al., "PulseRF: Physics augmented ML modeling and synthesis for high-frequency RFIC design," in *Proc. 43rd IEEE/ACM Int. Conf. Comput.-Aided Design*, Oct. 2024, pp. 1–9.
- [14] L. Francisco et al., "Design rule checking with a CNN based feature extractor," in *Proc. ACM/IEEE 2nd Workshop Mach. Learn. CAD (MLCAD)*, Nov. 2020, pp. 9–14.
- [15] A. Sohrabizadeh et al., "Automated accelerator optimization aided by graph neural networks," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 55–60.
- [16] A. Sohrabizadeh et al., "AutoDSE: Enabling software programmers to design efficient FPGA accelerators," *ACM Trans. Design Autom. Electron. Syst.*, vol. 27, no. 4, pp. 1–27, Jul. 2022.
- [17] A. Sohrabizadeh et al., "Robust GNN-based representation learning for HLS," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [18] Z. Qin et al., "Cross-modality program representation learning for electronic design automation with high-level synthesis," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–12.
- [19] Z. Ding et al., "Efficient task transfer for HLS DSE," in *Proc. 43rd IEEE/ACM Int. Conf. Computer-Aided Design*, Oct. 2024, pp. 1–9.
- [20] W. Li et al., "Hierarchical mixture of experts: Generalizable learning for high-level synthesis," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 17, p. 18.
- [21] J. Cong et al., "Source-to-source optimization for HLS," in *FPGAs for Software Programmers*. Cham, Switzerland: Springer, 2016, pp. 137–163.
- [22] H. Ye et al., "ScaleHLS: A new scalable high-level synthesis framework on multi-level intermediate representation," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Apr. 2022, pp. 741–755.
- [23] H. Ye, H. Jun, and D. Chen, "HIDA: A hierarchical dataflow compiler for high-level synthesis," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, vol. 1, Apr. 2024, pp. 215–230.
- [24] H. Chen et al., "Allo: A programming model for composable accelerator design," *Proc. ACM Program. Lang.*, vol. 8, pp. 593–620, Jun. 2024.
- [25] S. Basalama and J. Cong, "Stream-HLS: Towards automatic dataflow acceleration," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, Feb. 2025, pp. 103–114.
- [26] S. Pouget, L.-N. Pouchet, and J. Cong, "A unified framework for automated code transformation and pragma insertion," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, Feb. 2025, pp. 187–198.
- [27] Y. Fu et al., "GPT4AIGChip: Towards next-generation AI accelerator design automation via large language models," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [28] Y. Zhang et al., "MG-verilog: Multi-grained dataset towards enhanced LLM-assisted verilog generation," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [29] Y. Bai et al., "Towards a comprehensive benchmark for high-level synthesis targeted to FPGAs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 45288–45299.
- [30] L. J. Wan et al., "An iteratively-refined dataset for high-level synthesis functional verification through LLM-aided bug injection," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–6.
- [31] Z. Yu et al., "Invited paper: LLM4HWDesign contest: Constructing a comprehensive dataset for LLM-assisted hardware code generation with community efforts," in *Proc. 43rd IEEE/ACM Int. Conf. Comput.-Aided Design*, Oct. 2024, pp. 1–5.
- [32] H. M. Makrani et al., "Pyramid: Machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design," in *Proc. 29th Int. Conf. Field Program. Log. Appl. (FPL)*, Sep. 2019, pp. 397–403.
- [33] L. Ni et al., "OpenLS-DGF: An adaptive open-source dataset generation framework for machine learning tasks in logic synthesis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 10, pp. 3830–3843, Oct. 2025.
- [34] A. Hosny et al., "DRiLLS: Deep reinforcement learning for logic synthesis," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 581–586.
- [35] G. Pasandi, S. Pratty, and J. Forsyth, "AISYN: AI-driven reinforcement learning-based logic synthesis framework," 2023, *arXiv:2302.06415*.
- [36] W. L. Neto et al., "LSOracle: A logic synthesis framework driven by artificial intelligence: Invited paper," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2019, pp. 1–6.
- [37] A. B. Chowdhury et al., "INVICTUS: Optimizing Boolean logic circuit synthesis via synergistic learning and search," 2023, *arXiv:2305.13164*.
- [38] N. Wu et al., "Gamora: Graph learning based symbolic reasoning for large-scale Boolean networks," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.
- [39] H. Foster. (Feb. 2025). *IC/ASIC Functional Verification Trend Report-2024*. Siemens EDA and Wilson Research Group. [Online]. Available: <https://verificationacademy.com/topics/planning-measurement-and-analysis/wrg-industry-data-and-trends/2024-siemens-eda-and-wilson-research-group-ic-asic-functional-verification-trend-report/>
- [40] S. Chattopadhyay et al., "G-QED: Generalized QED pre-silicon verification beyond non-interfering hardware accelerators," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.
- [41] S. Mitra et al., "Silent data corruption by 10× test escapes threatens reliable computing," *IEEE Design Test*, vol. 42, no. 6, pp. 40–53, Dec. 2025.
- [42] E. Cheng et al., "CLEAR cross-layer resilience: A retrospective," *IEEE Design Test*, vol. 42, no. 3, pp. 74–85, Jun. 2025.
- [43] J. Blocklove et al., "Chip-chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE 5th Workshop Mach. Learn. CAD (MLCAD)*, Sep. 2023, pp. 1–6, doi: 10.1109/MLCAD58807.2023.10299874.
- [44] K. Chang et al., "ChipGPT: How far are we from natural language hardware design," 2023, *arXiv:2305.14019*.
- [45] A. Nakkab et al., "Rome was not built in a single step: Hierarchical prompting for LLM-based chip design," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–11.
- [46] Y. Lu et al., "RTLML: An open-source benchmark for design RTL generation with large language model," in *Proc. 29th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2024, pp. 722–727.
- [47] M. Liu et al., "VerilogEval: Evaluating large language models for Verilog code generation," 2023, *arXiv:2309.07544*.
- [48] C.-T. Ho, H. Ren, and B. Khailany, "VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 1, pp. 300–307.
- [49] K. Tasnia and S. Rahman, "OPL4GPT: An application space exploration of optimal programming language for hardware design by LLM," in *Proc. 30th Asia South Pacific Design Autom. Conf.*, Jan. 2025, pp. 981–987.
- [50] S. Mitra, "New verification unlocks AI-boosted chip design," presentation at the NSF Workshop AI Electron. Design Automat., Vancouver, BC, Canada, Dec. 2024.
- [51] V. Ganesh, S. A. Seshia, and S. Jha, "Machine learning and logic: A new frontier in artificial intelligence," *Formal Methods Syst. Des.*, vol. 60, no. 3, pp. 426–451, Jun. 2022.
- [52] J. H. Liang et al., "Exponential recency weighted average branching heuristic for SAT solvers," in *Proc. 13th AAAI Conf. Artif. Intell.*, 2016, pp. 3434–3440.
- [53] P. Jha et al., "RLSF: Fine-tuning LLMs via symbolic feedback," 2024, *arXiv:2405.16661*.